

PROJECT TASK LIST

Autonomous Multi-Agent Research System

vLLM · HuggingFace · LangGraph · FastAPI · MCP · SQLite · Streamlit

PROJECT PHASES

- Weeks 1–3** Local development — no GPU, no RunPod, no cost
- Week 4** RunPod provisioning — vLLM setup, models deployed
- Weeks 5–6** Agent development — single agent then multi-agent
- Week 7** UI & streaming — demo-ready frontend
- Week 8** Eval & polish — portfolio-ready launch

TICKET TYPES

- INFRA** Cloud & infrastructure
- SETUP** Scaffolding & config
- FEATURE** New functionality
- RELIABILITY** Error handling & resilience
- TESTING** Tests & quality
- BACKEND** API & server-side
- FRONTEND** UI & client-side
- EVAL** Benchmarking
- DOCS** Documentation & ADRs
- MCP** MCP protocol work
- DATABASE** Schema & persistence

PRIORITY High = blocking Med = important Low = nice to have *ESTIMATE* focused hours, add 30% buffer for debugging

Week 1 — Project Scaffold & Local Dev Environment ■ No vLLM needed

 Goal: Full repo structure in place, Python env running, mock LLM client ready so all future work can proceed without RunPod

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W1-01	SETUP	Initialise git repo & folder structure Create the full directory layout: agent/, api/, ui/, deploy/, mcp/, notebooks/, docs/decisions/. Add .gitignore, .env.example, README skeleton.	✓ Repo pushed to GitHub with correct structure ✓ README has title, tech stack badges, and placeholder sections ✓ .env.example covers all env vars used in the project ✓ .gitignore excludes .env, __pycache__, *.pyc, .DS_Store	High	1h
W1-02	SETUP	Configure Python environment & dependencies Create pyproject.toml with all dependencies pinned: LangGraph, LangChain, FastAPI, uvicorn, httpx, pymupdf, python-dotenv, tavily-python, huggingface-hub. Separate dev group for pytest, ruff, black.	✓ pip install -e . completes cleanly on Python 3.11+ ✓ No version conflicts between LangChain and LangGraph ✓ ruff and black configured via pyproject.toml ✓ make lint and make test commands work from root	High	1h
W1-03	SETUP	Build mock LLM client Write a MockLLMClient that returns scripted responses from a fixture file. This lets every other component be built and tested without any GPU or API costs.	✓ MockLLMClient implements same interface as real vLLM client ✓ Responses loaded from tests/fixtures/mock_responses.json ✓ Supports streaming mode (yields tokens one at a time) ✓ Swap between mock and real via LLM_BACKEND env var	High	1h
W1-04	SETUP	Write vLLM client wrapper (targeting mock first) Write the vLLM HTTP client wrapper against the OpenAI-compatible API spec. Point it at the mock during dev. Same code will point at RunPod later — no changes needed.	✓ Client reads base_url, api_key, model_name from .env ✓ Supports chat completions with streaming and non-streaming ✓ Timeout and retry (3x exponential backoff) implemented ✓ Unit test: mock client returns expected fixture response	High	1h
W1-05	INFRA	Scaffold Docker Compose stack Write Dockerfiles for api/ and ui/ services. Wire with docker-compose.yml so the full local stack starts with one command. Services can be stubbed — just needs to boot without errors.	✓ docker-compose up starts both api and ui containers ✓ Services read config from .env file via env_file ✓ No hardcoded ports — all configurable via env vars ✓ README documents the one-	Med	2h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
			line startup command		
W1-06	SETUP	Set up CI pipeline (GitHub Actions) Add a GitHub Actions workflow that runs lint (ruff), type check (mypy), and tests (pytest) on every push to main. Keeps the repo clean throughout the project.	✓ Workflow triggers on push and pull_request to main ✓ ruff, mypy, and pytest all run in sequence ✓ Failing tests block merge ✓ Workflow completes in under 2 minutes	Low	1h

Week 2 — Tool Layer (All 4 Tools, Mock-Tested) ■ No vLLM needed

 Goal: All four agent tools fully implemented, independently tested with mocks, and ready to plug into the agent

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W2-01	FEATURE	Build Tavily web search tool Integrate Tavily Search API. Returns structured results (title, url, snippet). Handles rate limits, empty results, query length limits, and timeouts with a clean ToolError schema.	✓ Returns list of SearchResult(title, url, snippet, score) ✓ Snippets truncated to 500 chars to protect context window ✓ Handles rate limit (429) with exponential backoff ✓ Returns ToolError on failure — never raises uncaught exception ✓ Unit tests: success path, empty results, rate limit, timeout	High	3h
W2-02	FEATURE	Build PDF parser tool Fetch PDFs from URL or local path using pymupdf. Extract text, chunk at ~500 tokens with 50-token overlap, score chunks against query by keyword overlap, return top-3.	✓ Fetches PDFs from HTTP URLs and local paths ✓ Chunks text at 500 words with 50-word overlap ✓ Returns top-3 chunks scored by keyword overlap with query ✓ Detects scanned PDFs (no text layer) and returns clear error ✓ Handles password-protected PDFs gracefully ✓ Page limit capped at 30 to prevent hangs	High	4h
W2-03	FEATURE	Build sandboxed code executor tool Run agent-generated Python snippets in a subprocess with a 10-second timeout. Capture stdout and stderr. Block dangerous imports. Apply resource limits.	✓ Runs code in isolated subprocess, 10s timeout enforced ✓ stdout and stderr both captured and returned ✓ Dangerous imports (os.system, subprocess, socket) blocked ✓ Code length capped at 2000 chars ✓ Returns ToolError with output on timeout or failure ✓ Unit tests: success, timeout, blocked import, syntax error	High	4h
W2-04	FEATURE	Build HuggingFace Hub search tool Query HuggingFace Hub API to find models and datasets by task or keyword. Returns top-5 results with name, description, downloads, and URL.	✓ Searches both /api/models and /api/datasets endpoints ✓ Filters by pipeline_tag when task type specified ✓ Returns top-5 results with metadata	High	2h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
			✓ API token loaded from env var (optional — works without it) ✓ Unit tests: model search, dataset search, API error		
W2-05	TESTING	Integration tests for all tools Write pytest integration tests for every tool covering success and failure paths. All tests use mocked HTTP calls — no real API keys or internet required.	✓ Each tool has at least 3 test cases ✓ All HTTP calls mocked with pytest-httpx or responses library ✓ Tests run in under 20 seconds with no external calls ✓ No API keys or secrets in test code ✓ Coverage report shows >80% on tools.py	High	3h
W2-06	RELIABILITY	Standardise ToolError schema across all tools Define a ToolError dataclass with code, message, retryable flag, and tool name. All tools raise ToolError consistently. Agent layer will catch and handle these.	✓ ToolError has: code (str), message (str), retryable (bool), tool_name (str) ✓ All 4 tools raise ToolError on failure (never bare exceptions) ✓ Retryable=True on network timeouts and rate limits ✓ Retryable=False on invalid input and auth errors ✓ Unit test verifies each tool raises correct ToolError types	High	2h

Week 3 — MCP SQLite Server, DB Agent & FastAPI Skeleton ■ No vLLM needed

🎯 Goal: MCP server running locally, database agent wired up, and FastAPI skeleton serving mock responses end-to-end

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W3-01	DATABASE	Design and create SQLite schema Define schema for message_history (session_id, role, content, timestamp), token_usage (session_id, agent_name, prompt_tokens, completion_tokens, cost_eur, timestamp), and sessions tables.	✓ Schema in db/schema.sql with CREATE TABLE statements ✓ Indexes on session_id for fast lookups ✓ Migration script applies schema to fresh DB ✓ Schema documented in docs/ with field descriptions ✓ Unit test: schema creates without errors, basic insert/select works	High	2h
W3-02	MCP	Build MCP SQLite server Implement a local MCP server using stdio transport that exposes tools: save_message, get_history, save_token_usage, get_session_stats, clear_session. The DB agent will call these via JSON-RPC.	✓ MCP server starts with python mcp/sqlite_server.py ✓ Exposes 5 tools with correct JSON-RPC schemas ✓ save_message writes to message_history table ✓ get_history returns all messages for a session_id ✓ save_token_usage writes cost and token counts ✓ All tools return structured JSON responses	High	4h
W3-03	MCP	Write MCP client wrapper for agent use Write a Python client that the DB agent uses to call the MCP server over stdio. Abstracts the JSON-RPC protocol behind clean method calls.	✓ MCPClient.save_message(), get_history(), save_token_usage() all work ✓ Client starts and manages MCP server subprocess lifecycle ✓ Handles MCP server startup errors gracefully ✓ Unit tests mock the MCP subprocess and verify correct JSON-RPC calls	High	2h
W3-04	FEATURE	Build database agent Create the DB agent as a LangGraph node that uses the MCP client to persist message history and token usage. Called by the supervisor at the start and end of every session.	✓ DB agent saves all inter-agent messages to SQLite via MCP ✓ Saves token usage after each LLM call ✓ Can retrieve session history on request ✓ Handles MCP server unavailability gracefully (logs	High	3h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
			warning, continues) ✓ Integration test: full save + retrieve cycle via MCP		
W3-05	BACKEND	Build FastAPI skeleton with mock LLM Implement /query, /history/{session_id}, /health, and /stats endpoints wired to MockLLMClient. Full request/response cycle works end-to-end before vLLM exists.	✓ /query accepts {question, session_id} and returns mock agent response ✓ /history returns list of past exchanges from SQLite ✓ /health returns status, model_name (mock), uptime ✓ /stats returns total tokens and cost (all zero with mock) ✓ FastAPI docs auto-generated at /docs	High	3h
W3-06	BACKEND	Add request validation and rate limiting Validate all incoming requests with Pydantic v2 models. Add basic rate limiting (10 req/min per IP) to prevent abuse during the demo.	✓ Pydantic models validate all request bodies ✓ Invalid payloads return 422 with clear error messages ✓ Question field max 2000 chars enforced ✓ Rate limit: 10 req/min per IP, returns 429 with Retry-After header ✓ Tests cover validation failure and rate limit hit	Med	2h
W3-07	TESTING	End-to-end test: full stack with mock LLM Write a pytest integration test that boots the FastAPI app, sends a question, and verifies the response is saved to SQLite via MCP. Full stack, zero GPU cost.	✓ Test sends POST /query and gets a structured response ✓ Response is saved to message_history in SQLite ✓ GET /history returns the saved message ✓ Test runs in isolation (temp SQLite file, no shared state) ✓ Test completes in under 5 seconds	Med	2h

Week 4 — RunPod Provisioning & vLLM Deployment

 Goal: All three models live on RunPod, endpoints verified, and LLM client switched from mock to real

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W4-01	INFRA	Provision RunPod instance and install vLLM Spin up an RTX 3090 or 4090 on RunPod using a CUDA 12.1+ base template. Install vLLM via pip. Verify GPU is detected and vLLM imports cleanly.	✓ RunPod pod launches with GPU detected via nvidia-smi ✓ vLLM installs without errors ✓ python -c "import vllm" succeeds ✓ Setup steps documented in deploy/runpod_setup.sh ✓ HF_TOKEN stored as RunPod environment variable (not in code)	High	3h
W4-02	INFRA	Deploy Mistral-7B-Instruct (supervisor model) Pull Mistral-7B-Instruct-v0.3 from HuggingFace and serve on port 8000 via vLLM with OpenAI-compatible API. This is the supervisor and DB agent model.	✓ Model downloads and loads into VRAM without OOM error ✓ vLLM server starts on port 8000 ✓ curl test to /v1/chat/completions returns a valid response ✓ TTFT (time to first token) under 3s on a simple prompt ✓ Max model length and GPU memory utilisation configured	High	2h
W4-03	INFRA	Deploy Mistral-7B fast instance (search + doc agents) Serve a second Mistral-7B instance on port 8001 configured with lower temperature defaults and optimised for retrieval-style tasks.	✓ Second vLLM instance serves on port 8001 ✓ Default temperature set lower (0.1) for deterministic retrieval ✓ Both instances run simultaneously without memory conflict ✓ Endpoint verified via curl from MacBook	High	2h
W4-04	INFRA	Deploy DeepSeek-Coder-7B (code agent model) Pull DeepSeek-Coder-7B-Instruct from HuggingFace and serve on port 8002. Verify it produces valid Python code on a test prompt.	✓ DeepSeek-Coder-7B-Instruct loads and serves on port 8002 ✓ Test prompt returns syntactically valid Python code ✓ Model card and licence reviewed and noted in README ✓ Memory usage across all 3 instances within GPU limits	High	2h
W4-05	INFRA	Secure endpoints and configure networking Enable API key auth on all three vLLM endpoints. Set up port forwarding or secure tunnel so MacBook can reach all three	✓ All 3 endpoints require Authorization: Bearer <key> header ✓ Connection from MacBook verified for all 3 ports ✓ API keys stored in .env	High	2h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
		endpoints reliably.	(gitignored) on MacBook ✓ Latency test: all 3 endpoints respond within 5s from MacBook		
W4-06	INFRA	Write and test runpod_setup.sh provisioning script Transcribe all manual setup steps into deploy/runpod_setup.sh. Test on a fresh RunPod pod to confirm it reproduces the full environment from scratch.	✓ Script runs end-to-end on a fresh pod without manual intervention ✓ All 3 models are downloaded and served after script completes ✓ Script is idempotent (safe to run twice) ✓ Estimated runtime documented in script comments (~20–30 min for downloads)	High	2h
W4-07	SETUP	Switch LLM_BACKEND env var from mock to vLLM Update .env to point all three agent model slots at the real RunPod endpoints. Verify the full stack from Week 3 still passes with real LLM responses.	✓ LLM_BACKEND=vllm routes to RunPod in all agent configs ✓ LLM_BACKEND=mock still works for offline development ✓ /health endpoint returns all 3 model names correctly ✓ Week 3 end-to-end test passes with real vLLM responses	High	1h

Week 5 — Single Agent End-to-End (Real vLLM)

🎯 Goal: One fully working ReAct agent using real vLLM, all 4 tools live, conversation history saved via MCP

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W5-01	FEATURE	Build LangGraph ReAct agent graph Create the core LangGraph state graph with plan, tool_call, observe, and answer nodes. Wire to the real vLLM client (Mistral fast instance) and all 4 tools.	✓ Agent graph defined in agent/graph.py with typed state ✓ Plan node sends ReAct prompt and parses tool call or final answer ✓ Tool call node routes to correct tool by name ✓ Observe node feeds tool result back into state ✓ Answer node returns final response when agent is done ✓ Max 10 iterations enforced to prevent infinite loops	High	4h
W5-02	FEATURE	Write ReAct system prompt and tool descriptions Write the system prompt that instructs the model to reason step-by-step and call tools in the correct JSON format. Write clear docstrings for each tool — the model reads these.	✓ System prompt produces consistent Thought/Action/Observation format ✓ Tool descriptions are clear enough that model picks correct tool >80% of test queries ✓ Prompt version controlled in agent/prompts.py ✓ A/B test: 10 queries with two prompt variants, pick the better one	High	2h
W5-03	FEATURE	Wire DB agent into session lifecycle Call the DB agent at session start (load history) and after each agent turn (save messages and token usage to SQLite via MCP).	✓ Session history loaded from SQLite at start of each /query call ✓ Each message saved to SQLite via MCP after agent completes ✓ Token usage saved with agent_name, model, cost ✓ History correctly retrieved via GET /history/{session_id}	High	2h
W5-04	RELIABILITY	Add loop detection and recovery Detect when the agent calls the same tool with identical arguments twice in a row. Force a recovery step or early answer synthesis to prevent wasted GPU time.	✓ Repeated identical tool call detected in state ✓ After 2 repeats agent is forced to answer with what it has ✓ Loop events logged with full state snapshot ✓ Unit test covers stuck-loop scenario with mock LLM	High	2h
W5-05	RELIABILITY	Add token budget tracking per	✓ Token count extracted from every vLLM response	High	2h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
		session Track prompt + completion tokens on every LLM call. Enforce a per-session token budget limit to prevent runaway cost during demos and eval runs.	✓ Running total tracked in LangGraph state ✓ Session hard-stopped at 8000 tokens with a clear message ✓ Budget stats logged per turn for debugging		
W5-06	TESTING	Manual test: 10 research queries end-to-end Run 10 handpicked research questions through the single agent and document results. Mix single-tool and multi-tool queries. Fix any bugs found.	✓ All 10 queries return a substantive answer (not an error) ✓ At least 3 queries use 2+ tools in a chain ✓ Agent trace (tool calls + observations) logged for each ✓ Bugs found are filed as GitHub issues and fixed before W6	High	3h

Week 6 — Multi-Agent Orchestration (Supervisor + 4 Specialists)

🎯 Goal: Full multi-agent system working: supervisor routes to all 4 specialist agents via LangGraph handoffs

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W6-01	FEATURE	Build supervisor agent graph Create the supervisor LangGraph graph using Mistral-7B (supervisor instance). It receives the query, makes a plan, delegates to specialist agents via handoffs, and synthesises the final answer.	✓ Supervisor graph defined in agent/supervisor.py ✓ Routes to correct specialist(s) for at least 80% of test queries ✓ Receives specialist observations via shared message channel ✓ Synthesises final answer from all observations ✓ Max 3 delegation rounds before forcing final answer	High	4h
W6-02	FEATURE	Promote single agent to Search + Document specialist agents Refactor the Week 5 single agent into two specialist graphs: SearchAgent (Tavily + HuggingFace tools only) and DocumentAgent (PDF tool only), each with a specialised system prompt.	✓ SearchAgent only has access to web_search and huggingface_hub tools ✓ DocumentAgent only has access to pdf_parser tool ✓ Each agent has a specialised prompt improving task-specific performance ✓ Both agents return structured observations to the supervisor	High	3h
W6-03	FEATURE	Build Code specialist agent Create the CodeAgent graph wired to DeepSeek-Coder-7B. Specialised for writing, running, and debugging Python snippets. Returns code output as an observation.	✓ CodeAgent uses DeepSeek-Coder-7B endpoint ✓ Agent can write, execute, and iterate on code in one turn ✓ Returns both the code and its output as structured observation ✓ Falls back gracefully if code execution fails after 2 attempts	High	3h
W6-04	FEATURE	Implement LangGraph shared message channel Set up the shared state object that all agents read from and write to. Supervisor always has visibility of the full channel. DB agent persists channel contents to SQLite.	✓ Shared channel implemented as typed LangGraph state ✓ All agent observations written to channel after completion ✓ Supervisor reads full channel when synthesising answer ✓ DB agent saves complete channel state to SQLite at end of session	High	2h
W6-05	FEATURE	Implement agent handoffs and result aggregation	✓ Handoff sends task description + relevant context to specialist ✓ Specialist returns observation in	High	2h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
		Implement the mechanism by which the supervisor delegates a subtask to a specialist and waits for its observation before deciding what to do next.	a standard schema ✓ Supervisor can delegate to multiple specialists sequentially ✓ Parallel delegation documented as a future improvement in README		
W6-06	RELIABILITY	Add per-agent error handling and fallback If a specialist agent fails (tool error, LLM timeout, loop), the supervisor is notified with a structured error observation and can either retry or continue without that agent.	✓ Specialist failure returns AgentError(agent_name, reason, retryable) ✓ Supervisor retries retryable failures once ✓ Non-retryable failures logged and excluded from synthesis ✓ Final answer notes which agents failed if relevant	Med	2h
W6-07	TESTING	Integration tests: multi-agent routing Write tests verifying that the supervisor routes correctly to each specialist for different query types. Use mock LLM responses to keep tests fast.	✓ Code question routes to CodeAgent ✓ Search question routes to SearchAgent ✓ PDF URL in query routes to DocumentAgent ✓ Multi-part question delegates to 2+ specialists ✓ All tests run with mocked LLM — no RunPod cost	High	3h

Week 7 — Streaming, UI & Production Polish

 Goal: Demo-ready application with live streaming, reasoning trace panel, and Docker Compose running the full stack

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W7-01	BACKEND	Implement SSE streaming for agent trace Stream the agent's reasoning trace token-by-token using Server-Sent Events. Events include: token, tool_start, tool_end, agent_handoff, final_answer types.	✓ FastAPI StreamingResponse yields SSE events ✓ Events: {type: "token" "tool_start" "tool_end" "handoff" "final_answer", data: ...} ✓ Client receives events in real time as agent runs ✓ Connection closes cleanly with a "done" event when agent finishes ✓ Tested with curl --no-buffer	High	3h
W7-02	BACKEND	Add cost tracking endpoint and per-query breakdown Expose /stats endpoint returning total tokens, total cost (EUR), per-agent breakdown, and per-query breakdown. All data read from SQLite via MCP.	✓ /stats returns total_tokens, total_cost_eur, per_agent breakdown ✓ Per-query breakdown available via /stats?session_id=X ✓ Cost calculated from token counts + per-model pricing estimate ✓ Stats update after each completed query	High	2h
W7-03	FRONTEND	Build Streamlit chat interface Main chat UI: input field, submit button, streaming response display. Chat history shown for the session. Clear session button.	✓ User types question and sees response stream in token by token ✓ Chat history shows all exchanges in current session ✓ Clear session button resets history and starts new session_id ✓ UI remains responsive (no blocking) during streaming	High	3h
W7-04	FRONTEND	Build agent reasoning trace panel Collapsible side panel showing the full multi-agent trace in real time: which supervisor plan was made, which specialists were called, tool inputs/outputs, and how the answer was formed.	✓ Trace panel shows supervisor plan at top ✓ Each specialist delegation shown with agent name badge ✓ Tool calls shown with name, input, and output ✓ Trace updates in real time as SSE events arrive ✓ Panel is collapsible so it doesn't crowd the main chat	High	3h
W7-05	FRONTEND	Add session stats bar to UI Persistent bar at the bottom of the UI	✓ Stats bar updates after each query completes	Med	2h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
		showing live tokens used, estimated EUR cost, number of agent calls, and number of tool calls for the current session.	✓ Cost shown in EUR to 3 decimal places ✓ Agent and tool call counts shown ✓ Resets cleanly when session is cleared		
W7-06	INFRA	Finalise Docker Compose for full stack Ensure docker-compose.yml reliably starts api and ui services, mounts the SQLite db volume, and starts the MCP server as a sidecar process.	✓ docker-compose up starts all services cleanly ✓ MCP SQLite server starts as part of the api service ✓ SQLite DB persists between container restarts via volume mount ✓ docker-compose down --volumes cleans up completely ✓ README has one-liner startup command	High	2h
W7-07	RELIABILITY	Add stop_vllm.sh and graceful model swap script Write a script to gracefully stop vLLM processes on RunPod without terminating the pod, and a script to swap which model is loaded (to save VRAM between sessions).	✓ stop_vllm.sh gracefully terminates all 3 vLLM processes ✓ Swap script stops one model and starts another on the same port ✓ Both scripts documented in deploy/README.md ✓ Budget reminder printed when stop_vllm.sh is run	Med	2h

Week 8 — Evaluation, Portfolio Polish & Launch

🔗 Goal: Benchmark results published, production-quality README live, demo GIF recorded, repo public and shareable

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W8-01	EVAL	Design 20-question evaluation benchmark Create a fixed set of 20 research questions across 4 categories: single-tool (5), multi-tool (5), code-required (5), and multi-agent (5). Store as JSON with rubric.	✓ 20 questions in tests/fixtures/eval_questions.json ✓ Each question has expected_tools list and difficulty rating ✓ 4 categories evenly distributed ✓ Rubric documented: what counts as a correct answer per category	High	2h
W8-02	EVAL	Build automated evaluation harness eval.py script runs all 20 benchmark questions, scores answers using an LLM-as-judge (Mistral supervisor model), and outputs a timestamped JSON results file.	✓ python eval.py runs all 20 questions unattended ✓ LLM judge scores each answer 1–5 with written justification ✓ Results saved to eval_results/TIMESTAMP.json ✓ Summary table printed: score by category, pass/fail, cost per query ✓ --model flag allows testing different model configs	High	3h
W8-03	EVAL	Run eval: single-agent vs multi-agent comparison Run the benchmark against both the Week 5 single-agent config and the Week 6 multi-agent config. Record results and write a comparison analysis.	✓ Both configs evaluated on full 20-question set ✓ Results saved separately and compared in a markdown table ✓ Cost-per-query calculated for each config ✓ Analysis written in docs/eval_results.md ✓ Key finding included in README	Med	3h
W8-04	DOCS	Write production-quality README README should read like a technical blog post. Cover: what the project does, why the architecture choices were made, how to run it, eval results, and future work.	✓ Project explained in 2 sentences at the very top ✓ Architecture diagram (v2 PNG) embedded ✓ Quickstart: env to running in under 10 steps ✓ Eval results table with single vs multi-agent comparison ✓ Tech stack badges at top ✓ Future work section: parallel agents, embedding-based chunk scoring, fine-tuning	High	4h

ID	Type	Title & Description	Acceptance Criteria	Pri	Est
W8-05	DOCS	Record demo GIF / video Record a 90-second demo showing a multi-step question answered by the multi-agent system with the reasoning trace panel visible. Embed in README.	✓ Demo shows at least one supervisor → 2 specialist delegation chain ✓ Reasoning trace panel visible throughout ✓ At least one tool call shown expanding in the trace panel ✓ GIF or video embedded directly in README ✓ Resolution clear enough to read tool outputs	High	2h
W8-06	DOCS	Write Architecture Decision Records (ADRs) Write 4 short ADR docs explaining key design decisions. These signal senior-level thinking to interviewers.	✓ ADR-001: Why vLLM over serverless (reproducibility, portfolio value) ✓ ADR-002: Why LangGraph over raw LangChain agents (stateful, debuggable) ✓ ADR-003: Why MCP for SQLite (tool-based interface, modern pattern) ✓ ADR-004: Why SSE over WebSockets (simpler, stateless, sufficient) ✓ All ADRs in docs/decisions/, linked from README	Med	2h
W8-07	RELIABILITY	Security audit before making repo public Scan git history for accidentally committed secrets. Verify no API keys, tokens, or RunPod credentials are in any committed file.	✓ truffleHog or git-secrets scan returns no findings ✓ All .env files confirmed in .gitignore ✓ No hardcoded API keys or IPs in any source file ✓ SECURITY.md documents the threat model and how to report issues	High	1h
W8-08	DOCS	Make repo public and share Final check: repo is clean, README renders correctly on GitHub, all links work, demo GIF loads. Make public.	✓ README renders correctly on GitHub (images, badges, links) ✓ Demo GIF loads in README without clicking through ✓ All 4 ADRs linked and accessible ✓ Repo added to LinkedIn / CV / portfolio site ✓ GitHub topics tagged: vllm, langgraph, multi-agent, rag, python	Med	1h